

이인범

Backend Developer | 010-6800-8667 | robot082373@naver.com | inbeom.tistory.com

PROFESSIONAL SUMMARY

일 100만 건 이상의 보안 로그를 다루는 온프레미스 솔루션 환경에서 PostgreSQL 쿼리 튜닝과 Elasticsearch 기반의 대용량 집계, 조회 기능 구현으로 집계 파이프라인 성능을 30~50% 개선해 온 백엔드 개발자입니다. Docker 단독 운영 환경의 K3s 클러스터 전환 설계와 정제 모듈 통합 리팩토링, JUnit 기반 테스트 환경 구축과 커버리지 0→70% 확보로 이슈 발생률 30% 감소까지 시스템 전 계층의 실무 경험을 보유하고 있습니다. SK하이닉스, 한화생명, BC카드 등 대형 고객사 환경에서 데이터 정합성 이슈를 직접 트러블슈팅하며 운영 안정성 감각을 길러왔고, 서강대학교 AI-SW 대학원 과정을 병행하며 Claude Code 등 AI 도구를 적극 활용해 개발 생산성을 높이고 있습니다.

SKILLS

Backend : Java, Spring Boot, Spring MVC, MyBatis, JPA, JUnit

Database : PostgreSQL, Elasticsearch, OpenSearch, SQLite

Infra/DevOps : K3s, Docker, Jenkins, Prometheus, Grafana, pgBouncer

Frontend : Vue.js, Axios

Tools : Git (Git Flow), JasperReport, Claude Code

EXPERIENCES

WEEDS KOREA | 개발부문 | 백엔드 개발자 (대리) | 2023.06 ~ 재직 중

보안 솔루션 제품군의 백엔드 개발 및 대형 고객사(SK하이닉스, 한화생명, BC카드 등) 커스텀 개발과 유지보수. 대규모 로그 데이터 집계 파이프라인 설계와 최적화, RESTful API 개발, 사내 개발 인프라 현대화 담당.

- Discover (모니터링 코어)** : 집계 파이프라인 성능 튜닝, 데이터 처리 통합 모듈 설계, Vue.js 대시보드와 JasperReport 보고서
- Batch (통계 배치)** : Spring Boot 기반 스케줄링 모듈 구축, MART 테이블 설계와 파티셔닝, ES → OpenSearch 마이그레이션, JUnit 기반 배치 테스트로 신뢰성 확보
- Board (소명 워크플로우)** : RESTful API 설계, GS 인증 1등급 보안 강화, 금융권 커스텀 결제 라인 구현, 런타임 메모리 누수 분석과 해결
- 인프라 현대화** : K3s 클러스터 전환 설계, Prometheus와 Grafana 모니터링, pgBouncer 도입

EDUCATION

- 서강대학교 AI-SW 대학원 소프트웨어 전공 (석사) — 재학 중 (졸업 예정 2028)
- 청운대학교 컴퓨터공학과 (학사) — 2026 졸업
- 인하공업전문대학 컴퓨터시스템과 (전문학사) — 2024 졸업
- 한양공업고등학교 컴퓨터네트워크과 — 2021 졸업

CERTIFICATES & AWARDS

- 정보처리기사 (2025), 정보처리산업기사 (2023), 네트워크관리사 (2019), PC정비사 (2019)
- 과학기술정보통신부 주관 ICT 멘토링(KT) 수상
- 대한전자공학회 ICT 창의융합 캡스톤 디자인 경진대회 수상

EXPERIENCES IN DEPTH

01. PostgreSQL과 Elasticsearch 집계 파이프라인 성능 튜닝

PostgreSQL, Elasticsearch, pgBouncer, EXPLAIN ANALYZE

프로젝트 배경과 역할

일 수십만 건에서 100만 건 이상의 보안 로그 데이터에 대한 집계 쿼리 응답이 수 초 이상 지연됨. PostgreSQL과 Elasticsearch를 함께 사용하는 집계 파이프라인 전반에서 병목이 발생했고, 개발 서버의 잦은 커넥션 고갈로 인한 리소스 부족도 반복됨.

접근 방법

EXPLAIN ANALYZE로 Seq Scan과 집계 병목 구간을 식별하고, GROUP BY 기반 집계 쿼리를 윈도우 함수(OVER PARTITION BY)로 재작성하여 불필요한 전체 스캔을 제거함. PostgreSQL의 worker 프로세스 수와 work_mem을 운영 환경에 맞게 조정하고, ES는 인덱스 템플릿 재설계와 heap memory, bucket size 튜닝을 병행함. 커넥션 고갈 문제는 pgBouncer를 도입하여 session 모드 기반 커넥션 풀링으로 해결함. 경찰청 운영 환경에서는 약 15만 부서의 트리 조회가 1분 이상 지연되어 타임아웃이 발생함. 수만 회 반복되던 서브쿼리의 CTE 분리와 불필요한 역직렬화 제거, 재귀 쿼리 구조 개선까지 함께 적용해 단일 실행 구조로 정리함.

성과

- PostgreSQL과 Elasticsearch 양 스택 튜닝으로 전체 집계 성능 약 30~50% 개선
- 경찰청 트리 조회 응답 시간 1분 → 5초 이내(약 92% 단축)
- pgBouncer 도입 후 커넥션 고갈 장애 완전 해소, 개발 서버 CPU 및 Memory 부하 안정화

02. 데이터 처리 통합 모듈 설계와 리팩토링

Spring MVC, PostgreSQL, MyBatis, Vue.js

프로젝트 배경과 역할

데이터 가공과 정제 기능이 4개 모듈에 분산되고 파편화되어 있었음. 각 모듈이 독립적으로 정제 로직을 구현하다 보니 공통 로직 중복과 처리 흐름 불일치로 잦은 버그와 유지보수 비용이 발생함. 신규 기능 추가 시 4개 모듈을 모두 파악해야 했고, 한 곳만 수정해도 다른 모듈에서 사이드 이슈가 발생하는 구조였음.

접근 방법

데이터 흐름을 중심으로 4개 모듈을 단일 통합 모듈로 재설계함. 별도 브랜치를 분리하여 기존 코드에 영향 없이 처음부터 새로 설계하는 방식으로 진행함. 화면마다 개별 구현되어 있던 엑셀 다운로드 기능은 인터페이스와 추상클래스로 정의하고 팩토리 패턴을 적용하여 공통화함. 공통 응답 포맷(BaseResult)과 @ControllerAdvice 기반 예외 처리도 함께 표준화하여 프론트엔드 연계 비용을 줄임. MyBatis 쿼리 구조는 CTE와 UPSERT 패턴으로 개선하여 복잡한 상태 처리 로직을 DB 레이어로 위임함.

성과

- 데이터 정합성과 처리 흐름 일관성 향상, 신규 기능 추가 시 코드 변경 범위 대폭 감소
- 공통 응답 포맷 도입 이후 화면별 상이하던 응답 구조가 일관되어 프론트엔드 연계 오류 감소
- 신규 화면에 엑셀 다운로드 기능 추가 시 기존 코드 수정 없이 확장만으로 구현 가능

03. Spring Boot 스케줄링 모듈 기반 통계 시스템 구축과 검색엔진 마이그레이션

Spring Boot, PostgreSQL, OpenSearch

프로젝트 배경과 역할

대규모 로그 데이터 기반 통계와 집계 배치 처리의 성능 최적화와 신뢰성 확보가 필요했음. 기존 Elasticsearch 기반 검색 인프라의 라이선스와 확장성 한계로 OpenSearch로의 전환도 필요한 상황이었음.

접근 방법

Spring Boot 기반 스케줄링 모듈에서 통계 배치를 실행하고, 통계 데이터를 기반으로 DATA MART 구축하여 조회 성능을 최적화함. 테이블 파티셔닝을 적용하여 대규모 집계 처리 성능을 확보하고, Elasticsearch에서 OpenSearch로의 마이그레이션을 수행함. NoSQL 인덱스 매핑 과정에서 nested 타입의 복합 집계 제약을 해결하기 위해 주요 필드를 1depth keyword 타입으로 추가 매핑하여 조회 프로세스를 개선함. JUnit 기반 배치 테스트 환경을 구축하여 데이터 정합성 이슈를 사전에 검출할 수 있는 체계도 함께 마련함.

성과

- MART 테이블 구축으로 복잡한 통계 조회 쿼리의 응답 속도 크게 개선
- 파티셔닝 적용으로 배치 집계 성능 안정화
- OpenSearch 전환을 통해 라이선스 리스크 해소 및 NoSQL 집계 성능 향상

04. 대규모 데이터 집계 RESTful API 설계와 민감정보 암호화 적용

Spring MVC, PostgreSQL, Elasticsearch, ARIA-256, HMAC

프로젝트 배경과 역할

수십만에서 수백만 건의 NoSQL과 RDB 데이터를 집계하고 가공하여 대시보드에 제공해야 했음. 기존 API는 응답 포맷과 에러 처리 방식이 화면마다 상이하여 프론트엔드 협업 비용이 높았고, 금융권 고객사 환경에 맞는 민감정보 암호화 체계도 미비한 상태였음.

접근 방법

집계 조회 API, 데이터 정제 CRUD API, 결재 워크플로우 API 등 서비스 핵심 API를 설계하고 개발함. 내부 솔루션의 개인정보를 비롯한 민감정보 보호를 위해 ARIA-256 기반 암호화와 복호화 처리를 적용했고, HMAC 기반 행위 데이터 위변조 방지 로직도 함께 구현함. 에러 코드와 응답 포맷을 표준화하여 Vue.js 프론트엔드와의 협업 효율을 높였고, 대용량 집계 응답의 페이지네이션과 캐싱 전략도 함께 설계함. 관리자 권한 이력 추적은 Interceptor 레이어에서 API 요청을 가로채 자동 기록하는 방식으로 구현함. 현재는 삼성카드 암호화 확대 적용 프로젝트의 담당자로 진행 중임.

성과

- 삼성증권, 삼성카드 등 대형 금융권 고객사 환경에서 요구하는 민감정보 암호화 구조 확보
- GS 인증 1등급 기능, 보안 요구사항을 충족하는 서비스 안정성 확보
- 응답 포맷 표준화 이후 신규 API 개발 시 참고 기준이 생겨 협업 효율 개선

05. 사내 개발 서버 K3s 클러스터 전환 설계

K3s, Docker, MetalLB, Longhorn, Jenkins, Prometheus, Grafana, Loki

프로젝트 배경과 역할

물리 서버 4대를 독립적으로 운영하며 제품별로 컨테이너를 띄우거나 서비스 패키지를 직접 설치하는 방식이었음. 서버당 최대 10개 이상의 컨테이너가 올라가는 경우도 있었고, 4대 모두 CPU와 Memory 여유가 없는 상태에서 서버 간 리소스 분배가 안 되며 테스트 환경 간 격리도 되지 않는 문제가 있었음.

접근 방법

Docker 단독 운영 환경에서 K3s 클러스터로 전환하는 설계를 주도함. 마스터 1대와 워커 3대로 구성하고, 기존처럼 DB 등 파드에 직접 접근이 필요한 구조였기 때문에 Ingress Controller 대신 MetalLB를 사용하여 각 파드에 사설 IP를 부여하는 방식으로 설계함. 스토리지는 Longhorn으로 레플리카를 구성하여 데이터 유실을 방지함. Kubernetes Namespace를 활용한 환경 분리 전략과 사설 IP 기반 관리 체계를 수립하고, Prometheus와 Grafana 기반 리소스 모니터링 대시보드 구조를 설계함. Jenkins CI/CD 파이프라인을 정비하여 Git Flow 기반 버전 관리와 테스트 자동화 정책도 함께 수립함. Loki 기반 로그 수집까지 포함한 아키텍처를 설계했고, 실제 구축은 인프라팀

과 협업하여 진행함.

성과

- 클러스터 전환 설계안을 기반으로 인프라 현대화 로드맵 확립
- Namespace 기반 환경 격리로 컨테이너 간 간섭 문제 제거
- Git 브랜치 정책과 CI/CD 파이프라인 정비로 배포 프로세스 표준화 및 배포 사고 예방

06. 대형 고객사 운영 환경 장애 대응과 품질 체계 구축

PostgreSQL, Elasticsearch, JUnit, 트러블슈팅

프로젝트 배경과 역할

SK하이닉스 운영 환경에서 대규모 집계와 통계 데이터의 정합성 오류가 복합적으로 발생함. 개인정보 로그 기반 집계 서비스의 핵심 수치가 맞지 않는 크리티컬 이슈였고, 고객사 특화 커스텀 로직과 높은 컴포넌트 간 의존성으로 원인 특정이 어려운 상황이었음. 다수의 커스텀 로직과 레거시 코드가 혼재하여 사이드 이펙트도 빈번한 환경이었음.

접근 방법

개인정보 데이터 특성상 외부 반출이 불가하여 현장에 직접 방문해 트러블슈팅을 진행함. 컨테이너 로그, DB slow query 로그, ES 집계 응답 패턴을 교차 분석하며 오류 구간을 단계적으로 좁혀나감. 분석 결과 ES bucket size 미설정과 PostgreSQL 집계 쿼리 처리 순서 문제가 복합 작용함을 확인하고, ES 측은 bucket size 조정과 search_after 기반 페이징 처리를, PostgreSQL 측은 집계 쿼리 처리 순서 재정렬을 각각 적용하여 양 스택의 정합성을 함께 회복시킴. 동일 유형의 잠재 오류 구간을 일괄 점검하기 위한 전수조사도 함께 진행함. 이 경험을 바탕으로 JUnit 기반 단위와 통합 테스트 환경을 단계적으로 구축하고, 핵심 비즈니스 로직과 보안 처리 모듈부터 우선 적용하여 사이드 이펙트를 조기에 검출할 수 있는 체계를 마련함.

성과

- 현장에서 약 4시간 만에 근본 원인 특정 및 수정 완료, 동일 유형 이슈 재발 0건
- 주요 집계 항목별 점검 기준 문서화로 재발 방지 체계 확립
- 테스트 커버리지 0→70% 확보, 이슈 발생률 약 30% 감소
- 레거시 의존성이 복잡하게 얽힌 환경에서 코드 변경 시 발생하던 사이드 이펙트의 조기 검출 가능